5.2.59

Rushil Shanmukha Srinivas
EE25BTECH11057
Electrical Enggineering ,
IIT Hyderabad.

September 30, 2025

Problem Statement

Question : Solve the system of equations

$$2x + 3y + 3z = 5 \tag{2.1}$$
$$x - 2y + z = -4 \tag{2.2}$$
$$3x - y - 2z = 3 \tag{2.3}$$

Augmented Matrix and Gaussian Elimination

Solution:

The system of equations in matrix form is :

$$\begin{pmatrix} 2 & 3 & 3 \\ 1 & -2 & 1 \\ 3 & -1 & -2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 5 \\ -4 \\ 3 \end{pmatrix} \tag{3.1}$$

Forming the augmented matrix,

$$\left(\begin{array}{ccc|c} 2 & 3 & 3 & 5 \\ 1 & -2 & 1 & -4 \\ 3 & -1 & -2 & 3 \end{array}\right) \tag{3.2}$$

Using Gaussian elimination,

$$\left(\begin{array}{ccc|c} 2 & 3 & 3 & 5 \\ 1 & -2 & 1 & -4 \\ 3 & -1 & -2 & 3 \end{array}\right) \xleftarrow[R_2 \to 2R_2 - R_1]{R_3 \to 2R_3 - 3R_1} \left(\begin{array}{ccc|c} 2 & 3 & 3 & 5 \\ 0 & -7 & -1 & -13 \\ 0 & -11 & -13 & -9 \end{array}\right) \tag{3.3}$$

$$\left(\begin{array}{ccc|c} 2 & 3 & 3 & 5 \\ 0 & -7 & -1 & -13 \\ 0 & -11 & -13 & -9 \end{array}\right) \xleftrightarrow{R_3 \to R_3 - \frac{11}{7} R_2} \left(\begin{array}{ccc|c} 2 & 3 & 3 & 5 \\ 0 & -7 & -1 & -13 \\ 0 & 0 & \frac{-80}{7} & \frac{80}{7} \end{array}\right) \tag{3.4}$$

Using back substitution we get :

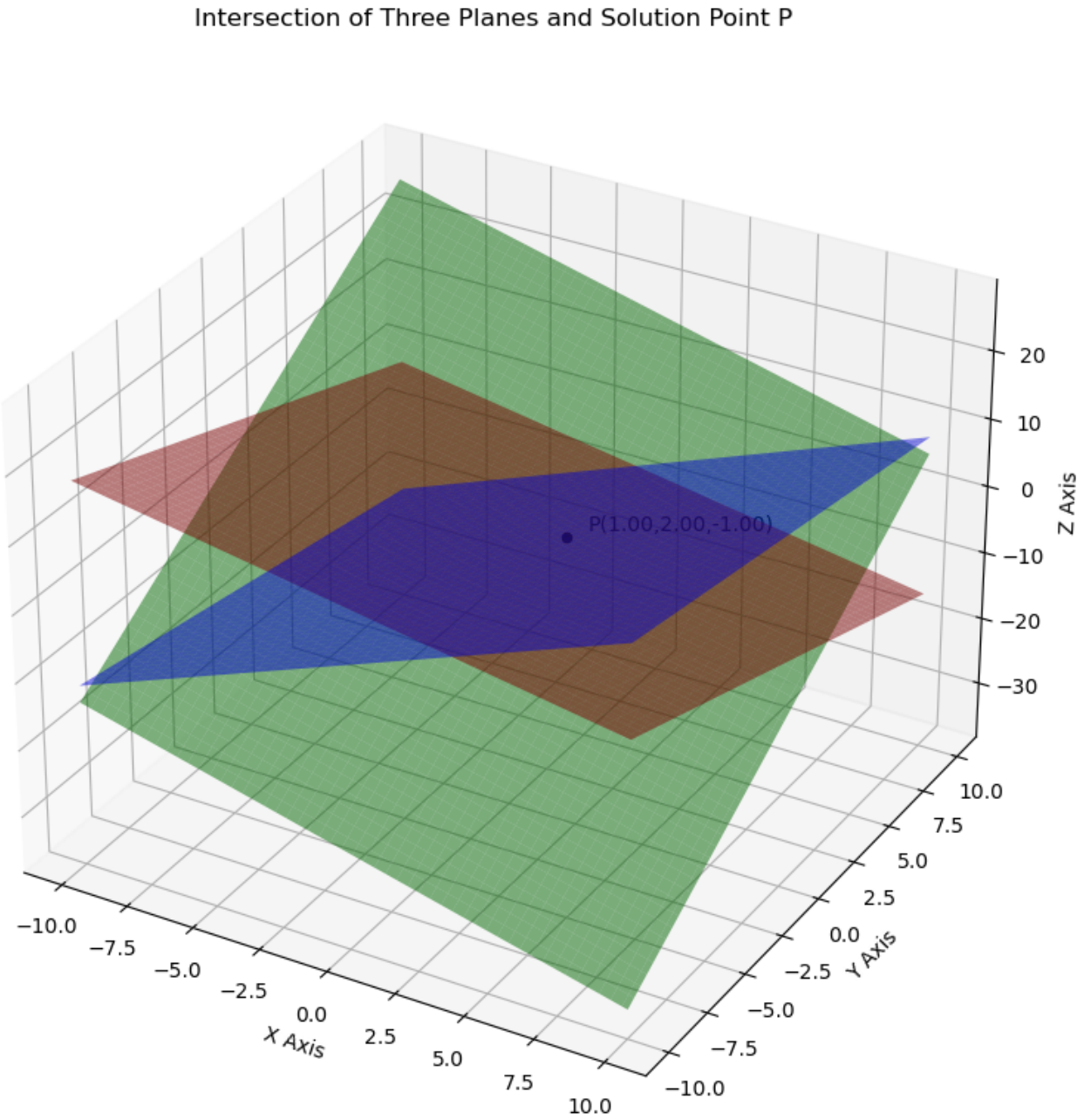
$$\frac{-80}{7} z = \frac{80}{7} \tag{3.5}$$
$$z = -1 \tag{3.6}$$
$$-7y - z = -13 \tag{3.7}$$
$$-7y + 1 = -13 \tag{3.8}$$
$$7y = 14 \tag{3.9}$$
$$y = 2 \tag{3.10}$$
$$2x + 3y + 3z = 5 \tag{3.11}$$
$$2x + 3 = 5 \tag{3.12}$$
$$x = 1 \tag{3.13}$$

Solution for System of Equations

Therefore the solution for the system of equations is :

$$\begin{pmatrix} 1 \\ 2 \\ -1 \end{pmatrix} \tag{3.14}$$

Plots


Intersection of Three Planes and Solution Point P
P(1.00,2.00,-1.00)
X Axis
Y Axis
Z Axis
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
20
10
0
−10
−20
−30

C Code

```c
#include <stdio.h>

// Function to compute determinant of a 3x3 matrix
double det3(double m[3][3]) {
    return m[0][0]*(m[1][1]*m[2][2] - m[1][2]*m[2][1])
         - m[0][1]*(m[1][0]*m[2][2] - m[1][2]*m[2][0])
         + m[0][2]*(m[1][0]*m[2][1] - m[1][1]*m[2][0]);
}

// Function to solve the system using Cramer's rule
// solution[0] -> x, solution[1] -> y, solution[2] -> z
void solve_system(double *solution) {
    double a[3][3] = {
        {2, 3, 3},
        {1, -2, 1},
        {3, -1, -2}
    };
    double b[3] = {5, -4, 3};

    double m[3][3];

    // Determinant of coefficient matrix
    double detA = det3(a);

    if(detA == 0) {
        solution[0] = solution[1] = solution[2] = 0.0;
        return;
    }

    // Dx (replace 1st column with b)
    for(int i=0;i<3;i++) for(int j=0;j<3;j++) m[i][j]=a[i][j];
    for(int i=0;i<3;i++) m[i][0] = b[i];
    double detX = det3(m);
```

```
    // Dy (replace 2nd column with b)
    for(int i=0;i<3;i++) for(int j=0;j<3;j++) m[i][j]=a[i][j];
    for(int i=0;i<3;i++) m[i][1] = b[i];
    double detY = det3(m);

    // Dz (replace 3rd column with b)
    for(int i=0;i<3;i++) for(int j=0;j<3;j++) m[i][j]=a[i][j];
    for(int i=0;i<3;i++) m[i][2] = b[i];
    double detZ = det3(m);

    solution[0] = detX / detA;
    solution[1] = detY / detA;
    solution[2] = detZ / detA;
}

// Optional main function (for direct C execution)
// This will be ignored when creating shared object
int main() {
    double solution[3];
    solve_system(solution);

    printf("Solution:\n");
    printf("x = %lf\n", solution[0]);
    printf("y = %lf\n", solution[1]);
    printf("z = %lf\n", solution[2]);

    return 0;
}
```

Python : call_c.py

```
import ctypes

# Load the shared object file
solver = ctypes.CDLL('./solve.so')

# Tell Python about the function signature
solver.solve_system.argtypes = [ctypes.POINTER(ctypes.c_double)]
solver.solve_system.restype = None

# Create an array of 3 doubles to hold the solution
solution = (ctypes.c_double * 3)()

# Call the C function
solver.solve_system(solution)

# Print the results
print("Solution from C shared object:")
print(f"x = {solution[0]}")
print(f"y = {solution[1]}")
print(f"z = {solution[2]}")
```

Python Code for Plotting

```
import os
import numpy as np
import matplotlib.pyplot as plt

# for saving figure in figs folder
figs_folder = os.path.join("..", "figs")

# Define system of equations
# 2*x +3*y +3*z = 5
# 1*x -2*y +1*z = -4
# 3*x -1*y -2*z = 3

A = np.array([[2, 3, 3],
              [1, -2, 1],
              [3, -1, -2]], dtype=float)

b = np.array([5, -4, 3], dtype=float)

# Solve using numpy
solution = np.linalg.solve(A, b)
x, y, z = solution
print("Solution from NumPy:", solution)

# Create meshgrid for plotting planes
x_vals = np.linspace(-10, 10, 100)
y_vals = np.linspace(-10, 10, 100)
X, Y = np.meshgrid(x_vals, y_vals)

# Plane 1: 2x + 3y + 3z = 5 -> z = (5 - 2x - 3y)/3
Z1 = (5 - 2*X - 3*Y)/3
```

```
# Plane 2: x - 2y + z = -4 -> z = (-4 -x + 2y)
Z2 =(-4 -X +2*Y)

# Plane 3: 3x - y - 2z = 3 -> z = (-3 + 3x - y )/2
Z3 = (-3 + 3*X - Y)/2

# Plotting
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection="3d")

# Plot the planes
ax.plot_surface(X, Y, Z1, alpha=0.5, color="red")
ax.plot_surface(X, Y, Z2, alpha=0.5, color="green")
ax.plot_surface(X, Y, Z3, alpha=0.5, color="blue")

# Plot the solution point
ax.scatter(x, y, z, color="black")
ax.text(x+0.5, y+0.5, z+0.5,
       f"P({x:.2f},{y:.2f},{z:.2f})",
       fontsize=10, color="black")

# Axes labels and title
ax.set_xlabel("X Axis")
ax.set_ylabel("Y Axis")
ax.set_zlabel("Z Axis")
ax.set_title("Intersection of Three Planes and Solution Point P")
ax.grid(True)

# Save figure
plt.tight_layout()
fig.savefig(os.path.join(figs_folder, "solution.png"))
plt.show()
```